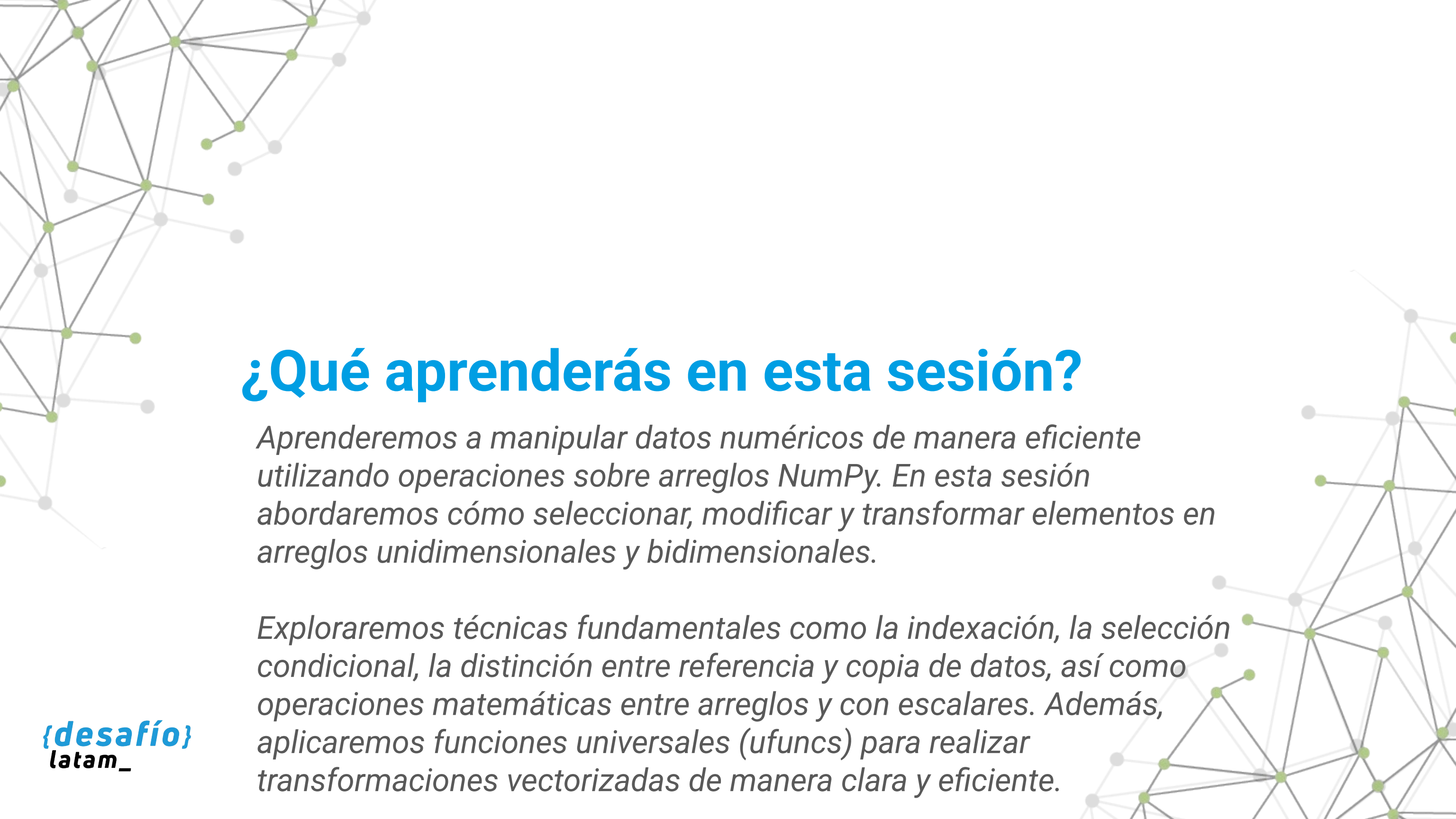




Obtención de datos y fundamentos de Numpy

Clase 5 - Operaciones con arreglos NumPy



¿Qué aprenderás en esta sesión?

Aprenderemos a manipular datos numéricos de manera eficiente utilizando operaciones sobre arreglos NumPy. En esta sesión abordaremos cómo seleccionar, modificar y transformar elementos en arreglos unidimensionales y bidimensionales.

Exploraremos técnicas fundamentales como la indexación, la selección condicional, la distinción entre referencia y copia de datos, así como operaciones matemáticas entre arreglos y con escalares. Además, aplicaremos funciones universales (ufuncs) para realizar transformaciones vectorizadas de manera clara y eficiente.

Desafío inicial

Trabajas en una empresa de logística que analiza rutas de reparto en tiempo real. Cada vehículo envía constantemente datos como velocidad, temperatura del motor, consumo de combustible y ubicación. Tu equipo necesita detectar patrones, identificar anomalías y preparar estos datos para análisis estadístico. Sin embargo, los archivos que recibes contienen miles de registros por hora, y trabajar con listas anidadas ha resultado lento y poco práctico.

Tu jefatura te pide reorganizar y procesar estos datos utilizando NumPy. Pero antes, debes dominar cómo seleccionar información relevante, filtrar condiciones específicas (como valores extremos), realizar cálculos entre columnas (como consumo por kilómetro) y aplicar funciones a gran escala, sin afectar el rendimiento del sistema.

**/* Indexación y selección: selección de
elementos de un arreglo. */**

¿Cómo manipular y transformar datos eficientemente?

¿Cómo podrías manipular y transformar estos datos de forma eficiente y sin errores?

Para resolver esta necesidad, deberás conocer técnicas de indexación, selección condicional, operaciones vectorizadas y funciones universales (ufuncs) en NumPy, asegurando un procesamiento rápido, claro y reutilizable para grandes volúmenes de datos.



Indexación y selección de elementos de un arreglo

¿Qué significa indexar un arreglo en NumPy?

En NumPy, indexar significa acceder a uno o varios elementos de un arreglo (ndarray) utilizando sus posiciones. Esta capacidad es esencial cuando necesitamos leer, modificar o extraer datos específicos para analizarlos o procesarlos. A diferencia de las listas tradicionales de Python, NumPy ofrece una variedad de formas para indexar datos que permiten seleccionar elementos de forma precisa y eficiente, tanto en arreglos unidimensionales (vectores) como en arreglos bidimensionales (matrices).

Indexación en arreglos unidimensionales

Supongamos que tienes un arreglo con datos de velocidad registrados por un sensor:

```
import numpy as np
velocidades = np.array([30, 45, 55, 60, 48, 52])
```

Acceso por posición

Podemos acceder a un valor individual indicando su índice (comienza en 0):

```
print(velocidades[0]) # 30
print(velocidades[3]) # 60
```

Índices negativos y rebanado

Índices negativos

También puedes usar índices negativos para contar desde el final:

```
print(velocidades[-1]) # 52 (último valor)
print(velocidades[-3]) # 60 (tercer valor desde
el final)
```

Rebanado o slicing

Si necesitas extraer una porción del arreglo:

```
segmento = velocidades[1:4]
print(segmento) # [45 55 60]
```

¿Qué ocurre si usas velocidades[:3] o velocidades[3:]?

- **[:3] selecciona los tres primeros valores.**
- [3:] selecciona desde la posición 3 hasta el final.

Indexación en arreglos bidimensionales (matrices)

Cuando trabajamos con matrices (dos dimensiones), la indexación requiere dos coordenadas: fila y columna.

Observemos:

```
matriz = np.array([[10, 20, 30],[40, 50, 60],[70, 80, 90]])
```


Acceso a elementos en matrices

Acceso a un elemento específico

```
print(matriz[1, 2]) # 60 (segunda fila, tercera columna)
```

Acceso a una fila completa

```
print(matriz[0]) # [10 20 30]
```

Acceso a una columna completa

```
print(matriz[:, 1]) # [20 50 80] → segunda columna de todas las filas
```

Modificación de valores mediante indexación

Podemos asignar nuevos valores usando índices:

```
velocidades[2] = 50  
print(velocidades) # [30 45 50 60 48 52]
```

En matrices:

```
matriz[2, 1] = 0  
print(matriz)  
# [[10 20 30]  
# [40 50 60]  
# [70 0 90]]
```

Reemplazo de secciones y acceso avanzado

Reemplazo de secciones completas

Es posible reemplazar un segmento completo del arreglo con nuevos valores:

```
velocidades[0:3] = [35, 46, 51]  
print(velocidades) # [35 46 51 60 48 52]
```

¿Qué pasaría si el número de elementos a reemplazar no coincide?

NumPy lanzará un error. Las secciones deben coincidir en forma y dimensión.

Acceso con listas de índices

Puedes seleccionar posiciones específicas usando una lista:

```
posiciones = [0, 2, 4]  
print(velocidades[posiciones]) # [35 51 48]
```

Esto se denomina indexación avanzada y es útil para aplicar filtros personalizados.

Indexación booleana

Aunque se profundizará más adelante, es importante saber que NumPy permite usar condiciones como índice para seleccionar datos que cumplan ciertos criterios.

Ejemplo:

```
print(velocidades[velocidades > 50])  
# Resultado: [51 60 52]
```

Buenas prácticas y advertencias



Compatibilidad de tipos

Al modificar elementos, asegúrate de que el tipo de dato sea compatible (por ejemplo, no asignar texto a un arreglo numérico).



Verificar dimensiones

Siempre revisa `.shape` y `.ndim` para confirmar las dimensiones del arreglo.



Evitar errores

La indexación incorrecta puede llevar a errores difíciles de detectar en procesamiento de datos.

¡Muy bien! Ya sabemos cómo acceder y modificar datos usando índices, es momento de avanzar hacia una herramienta aún más poderosa: la selección condicional de elementos, que nos permite filtrar datos en base a condiciones lógicas. Esta técnica es clave para el análisis exploratorio, detección de anomalías o preparación de subconjuntos de datos para modelos predictivos.

**/* Selección condicional de elementos
de un arreglo. */**

Selección condicional de elementos de un arreglo

¿Qué significa seleccionar condicionalmente elementos en NumPy?

En análisis de datos, es común tener que filtrar los valores que cumplen una cierta condición: por ejemplo, seleccionar solo los valores mayores a un umbral, encontrar los elementos que cumplan múltiples criterios o extraer filas específicas de una matriz que cumplan con cierta lógica. NumPy facilita este proceso mediante una técnica llamada indexación booleana, que permite seleccionar subconjuntos de datos en base a expresiones lógicas.



Condiciones sobre arreglos unidimensionales

Supongamos que tienes un vector con registros de temperatura:

```
import numpy as np
temperaturas = np.array([22.5, 25.0, 19.8, 27.3, 21.0])
```


Aplicar una condición

Podemos generar un arreglo de valores True o False indicando si cada elemento cumple la condición:

```
condicion = temperaturas > 23  
print(condicion)  
# Resultado: [False True False True False]
```

Usar la condición como índice

```
print(temperaturas[condicion])  
# Resultado: [25. 27.3]
```

Expresión compacta

Este enfoque nos permite filtrar automáticamente los datos.

Podemos hacer todo en una sola línea:

```
print(temperaturas[temperaturas > 23])  
# Resultado: [25. 27.3]
```

¿Qué otras condiciones podríamos aplicar?

- Menor que 21: temperaturas[temperaturas < 21]
- Igual a 25: temperaturas[temperaturas == 25]
- Distinto de 22.5: temperaturas[temperaturas != 22.5]



Condiciones sobre matrices

También podemos aplicar condiciones a matrices completas. Supongamos:

```
matriz = np.array([[10, 20, 30],[5, 25, 35],[15, 18, 40]])
```

Obtener los elementos mayores a 20

```
print(matriz[matriz > 20])  
# Resultado: [30 25 35 40]
```

Esto devuelve un vector con todos los elementos que cumplen la condición.

Máscaras booleanas reutilizables

Las condiciones se pueden almacenar como variables para su reutilización:

```
mascara = matriz % 2 == 0 # elementos pares  
print(matriz[mascara]) # Resultado: [10 20 30 4 18 40]
```

Filtrado combinado con asignación

Podemos usar condiciones no solo para seleccionar, sino también para modificar:

```
temperaturas[temperaturas < 22] = 22  
print(temperaturas) # Resultado: [22.5 25. 22. 27.3 22. ]
```

Esto es útil en limpieza de datos, imputación de valores mínimos o normalización.

**/* Referencia y copia de arreglos
en NumPy. */**

Referencia y copia de arreglos en NumPy

¿Qué sucede cuando asignas un arreglo a otra variable? ¿Es una copia o una referencia?

En Python, las variables funcionan como referencias a objetos en memoria. Cuando trabajamos con arreglos en NumPy, es fundamental entender esta diferencia, ya que una asignación puede no crear una copia nueva, sino simplemente otro nombre que apunta al mismo arreglo original. Esto puede llevar a errores sutiles y difíciles de detectar si se modifican datos sin intención.

Asignación directa: referencia al mismo arreglo

Veamos el siguiente ejemplo:

```
import numpy as np = np.array([1, 2, 3])  
b = a # No se crea una copia, b apunta al  
mismo arreglo  
b[0] = 100  
print("a:", a) # Resultado: [100 2 3]  
print("b:", b) # Resultado: [100 2 3]
```

¿Qué ocurrió? Al modificar b, también se modificó a, porque ambas variables referencian exactamente el mismo objeto en memoria. Esto se denomina aliasing.

¿Cómo verificar si dos arreglos comparten memoria?

Podemos usar `np.shares_memory()` para confirmar si dos arreglos apuntan al mismo bloque de datos:

```
np.shares_memory(a, b)  
# True
```

Crear una copia real del arreglo

Si deseamos trabajar con una copia completamente independiente, debemos usar el método `.copy()`:

Ahora, `c` es una nueva estructura de datos, y los cambios en ella no afectan a `a`.

```
c = a.copy()
c[0] = 999
print("a:", a) # Resultado: [100 2 3]
print("c:", c) # Resultado: [999 2 3]
```

Comparación

b = a	No	Sí
b = a.copy()	Sí	No
b = a[:]	No*	Parcialmente**

* a[:] crea una vista superficial (shallow copy), que en muchos casos sigue compartiendo memoria.

** Dependerá del tipo de slicing y si se modifica parte del arreglo.

Tabla 1. Comparación

Fuente: Desafío Latam

Copias en matrices: el caso con slicing

```
matriz = np.array([[1, 2], [3, 4]])  
submatriz = matriz[:, 0]  
# Selecciona la primera columna  
submatriz[0] = 100  
print("matriz:\n", matriz)
```

Resultado:

```
[[100, 2], [3, 4]]
```

Esto sucede porque submatriz no es una copia independiente, sino una vista (view) sobre la matriz original.

Cómo hacer una copia segura de una subestructura

```
submatriz = matriz[:, 0].copy()
submatriz[0] = 200
print("matriz:\n", matriz)
# La matriz no se ve afectada
```

¿Por qué es importante esto?

En ciencia de datos, es común transformar, limpiar y reestructurar datos. Si no distinguimos entre copiar y referenciar, podríamos:

- Corromper el conjunto de datos original por accidente.
- Propagar errores silenciosos en análisis.
- Obtener resultados inconsistentes en modelos o simulaciones.

/* Operaciones entre arreglos. */

Operaciones entre arreglos

¿Cómo realizar cálculos entre estructuras de datos sin usar bucles?

Uno de los grandes aportes de NumPy al ecosistema de Python es su capacidad para realizar operaciones vectorizadas, es decir, aplicar cálculos directamente sobre arreglos completos, sin necesidad de iterar elemento por elemento. Esto no solo hace que el código sea más limpio y fácil de leer, sino que también aumenta enormemente la eficiencia computacional.

Operaciones elementales entre arreglos

Supongamos que tienes dos vectores de igual tamaño:

```
import numpy as np
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

Podemos realizar operaciones aritméticas directamente:

Estas operaciones se aplican elemento por elemento, y son mucho más rápidas que un bucle tradicional en Python.

¿Qué pasa si los tamaños no coinciden?

NumPy intentará aplicar una técnica llamada broadcasting. Veamos:

```
a = np.array([10, 20, 30])
b = np.array([1])
# arreglo con un solo valor
print(a + b)
# Resultado: [11 21 31]
```

Aquí, b se "expande" automáticamente para coincidir con la forma de a.

Este comportamiento es útil, pero debemos tener cuidado cuando los tamaños no son compatibles, ya que puede generar errores:

```
a = np.array([10, 20, 30])
b = np.array([1, 2])
# print(a + b) # Error: operands could not be
broadcast together
```

Operaciones entre matrices

Las mismas operaciones se aplican entre matrices del mismo tamaño:

```
m1 = np.array([[1, 2], [3, 4]])  
m2 = np.array([[10, 20], [30, 40]])  
print(m1 + m2) # Resultado: # [[11 22] # [33 44]]
```

También pueden realizarse combinaciones entre una matriz y un vector, si las dimensiones son compatibles (broadcasting):

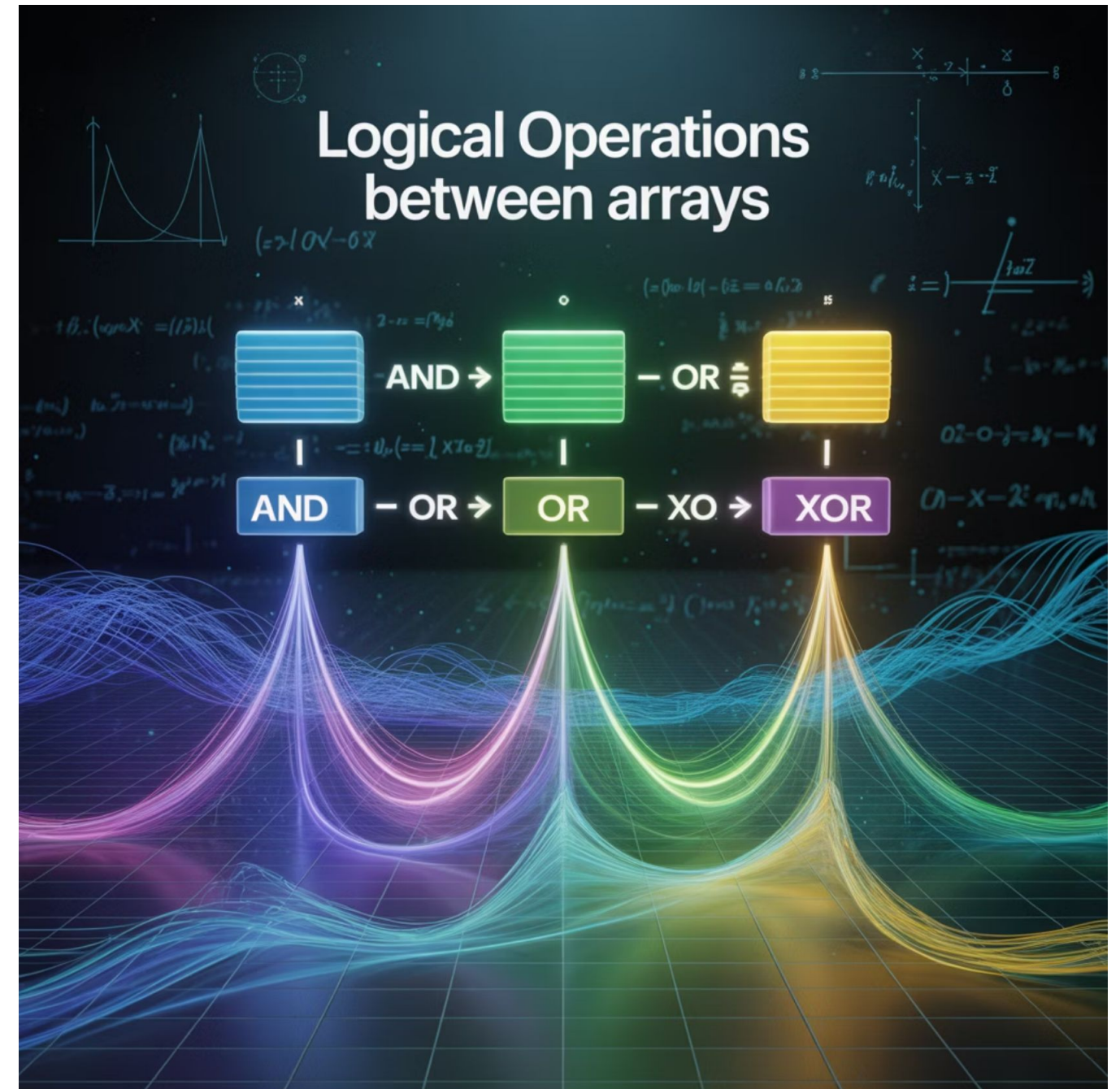
```
fila = np.array([1, 2])  
print(m1 + fila)  
# Resultado:  
# [[2 4]  
# [4 6]]
```

Operaciones lógicas y comparaciones

También puedes aplicar comparaciones directamente entre arreglos:

```
a = np.array([10, 20, 30])  
b = np.array([10, 15, 30])  
print(a == b) # [ True False True]  
print(a > b) # [False True False]
```

Esto devuelve un arreglo booleano, que puede ser usado para filtrar datos.



Operaciones combinadas y funciones

Podemos encadenar operaciones con funciones adicionales:

```
resultado = (a + b) / 2  
print(resultado)
```

O utilizar funciones de agregación de NumPy:

```
print(np.sum(a))  
# Suma total  
print(np.mean(a))  
# Promedio  
print(np.max(a))  
# Valor máximo  
print(np.std(a))  
# Desviación estándar
```


¿Qué pasa con los tipos de datos?

NumPy realiza promoción de tipos automáticamente cuando es necesario:

```
a = np.array([1, 2, 3])  
# int  
b = np.array([0.5, 1.5, 2.5])  
# float  
print(a + b)  
# Resultado: [1.5 3.5 5.5] → float
```

Esto asegura que los resultados mantengan la precisión adecuada.



/* Operaciones con escalares. */

Operaciones con escalares

¿Cómo puedes transformar todos los valores de un arreglo aplicando un mismo número?

En ciencia de datos es muy común aplicar operaciones uniformes a todos los elementos de una estructura. Esto puede incluir:

- Escalar todos los valores por un coeficiente.
- Sumar o restar un número fijo.
- Calcular proporciones o porcentajes.
- Aplicar constantes de normalización.

NumPy permite realizar estas tareas de forma directa gracias a la vectorización. Al operar un arreglo con un escalar, NumPy aplica automáticamente la operación a cada elemento del arreglo sin necesidad de bucles.

Suma y resta con escalares

Cada valor del arreglo es modificado con la operación indicada.

```
import numpy as np
datos = np.array([10, 20, 30])
print(datos + 5) # [15 25 35]
print(datos - 3) # [ 7 17 27]
```

Multiplicación y división con escalares

```
print(datos * 2)
# [20 40 60]
print(datos / 10)
# [1. 2. 3.]
```

Este enfoque es ideal para convertir unidades, calcular proporciones o ajustar magnitudes.

Operaciones de potenciación y módulo

```
print(datos ** 2)
# [100 400 900] → cuadrado
print(datos % 7)
# [3 6 2] → módulo 7
```

Estas operaciones también se aplican elemento por elemento.

Aplicaciones frecuentes



Ajuste de precios

```
precios = np.array([100, 200, 300])  
nuevos = precios * 1.05 # Aumento de 5%
```



Conversión de temperaturas

```
celsius = np.array([0, 10, 20, 30])  
fahrenheit = celsius * 9 / 5 + 32
```



Normalización de datos

```
datos = np.array([5, 10, 15])  
normalizados = datos / np.max(datos)  
# Escala entre 0 y 1
```

Compatibilidad automática

NumPy garantiza que este tipo de operaciones funcionen siempre que el escalar sea compatible con el tipo de dato del arreglo. Por ejemplo:

```
datos = np.array([5, 10, 15])
normalizados = datos / np.max(datos)
# Escala entre 0 y 1
```

NumPy convierte automáticamente el resultado si es necesario.

Buenas prácticas

- Antes de realizar operaciones, revisa el tipo de datos del arreglo con `.dtype`.
- Verifica si deseas mantener la precisión entera o permitir la conversión a float.
- Usa operaciones vectorizadas para mejorar el rendimiento de cálculos masivos.

Errores comunes

División entera sin esperarlo

```
a = np.array([5, 10])  
print(a / 2)  
# Resultado esperado: [2.5 5.0]
```

Pero si a fuera de tipo int32 y usas división entera //, perderás decimales:

```
print(a // 2)  
# Resultado: [2 5] → división truncada
```

Tipo de dato inesperado

```
a = np.array([True, False])  
print(a * 10)  
# Resultado: [10 0] → True = 1, False = 0
```

NumPy interpreta los valores booleanos como enteros.

Resumen visual

Suma	$arr + 3$	Cada valor + 3
Resta	$arr - 1$	Cada valor - 1
Multiplicación	$arr * 10$	Cada valor x 10
División	$arr / 5$	Cada valor / 5
Potencia	$arr ** 2$	Cada valor ²
Módulo	$arr \% 2$	Cada valor % 2

/* Aplicando funciones a un arreglo. */

Aplicando funciones a un arreglo

Avancemos para explorar el poder de aplicar funciones a un arreglo

NumPy

¿Cómo puedes calcular transformaciones y estadísticas sobre todos los elementos de un arreglo de forma rápida?

Una de las mayores fortalezas de NumPy es la posibilidad de aplicar funciones matemáticas y estadísticas directamente sobre arreglos, sin tener que escribir bucles. Esto permite procesar grandes volúmenes de datos con muy pocas líneas de código y con un rendimiento muy superior al de las listas tradicionales de Python.

Estas funciones están optimizadas en C y se ejecutan de forma vectorizada, es decir, se aplican automáticamente a todos los elementos del arreglo.

Funciones matemáticas elementales

NumPy incluye funciones básicas como:

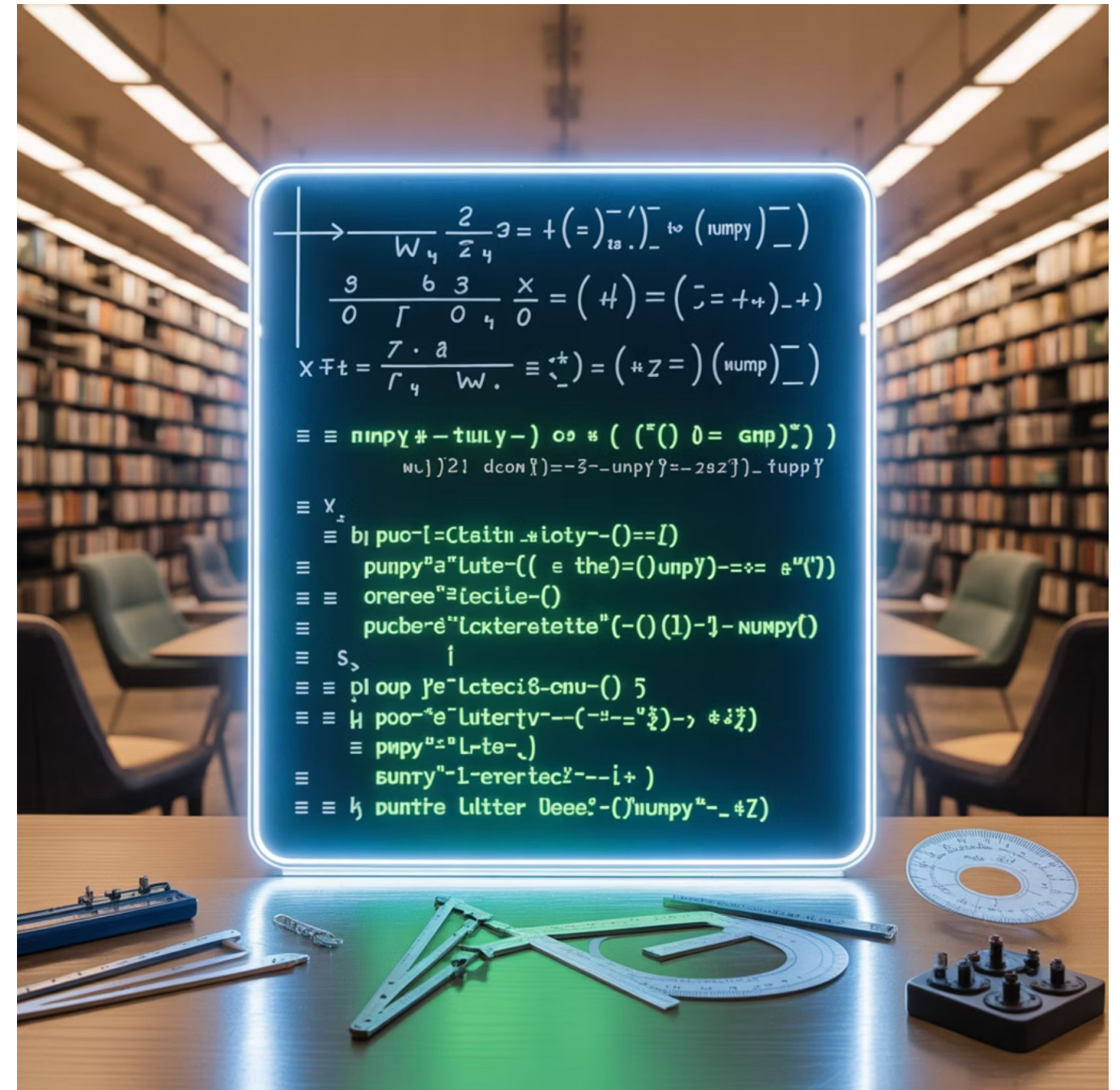
```
import numpy as np
datos = np.array([1, 4, 9, 16])
print(np.sqrt(datos))
# Raíz cuadrada: [1. 2. 3. 4.]

print(np.log(datos))
# Logaritmo natural

print(np.exp(datos))
# Exponencial e^x

print(np.sin(datos))
# Seno de cada elemento
```

Cada una de estas funciones transforma todos los elementos del arreglo de forma simultánea y sin bucles.



Aplicar funciones estadísticas

Estas funciones son esenciales para el análisis exploratorio de datos.

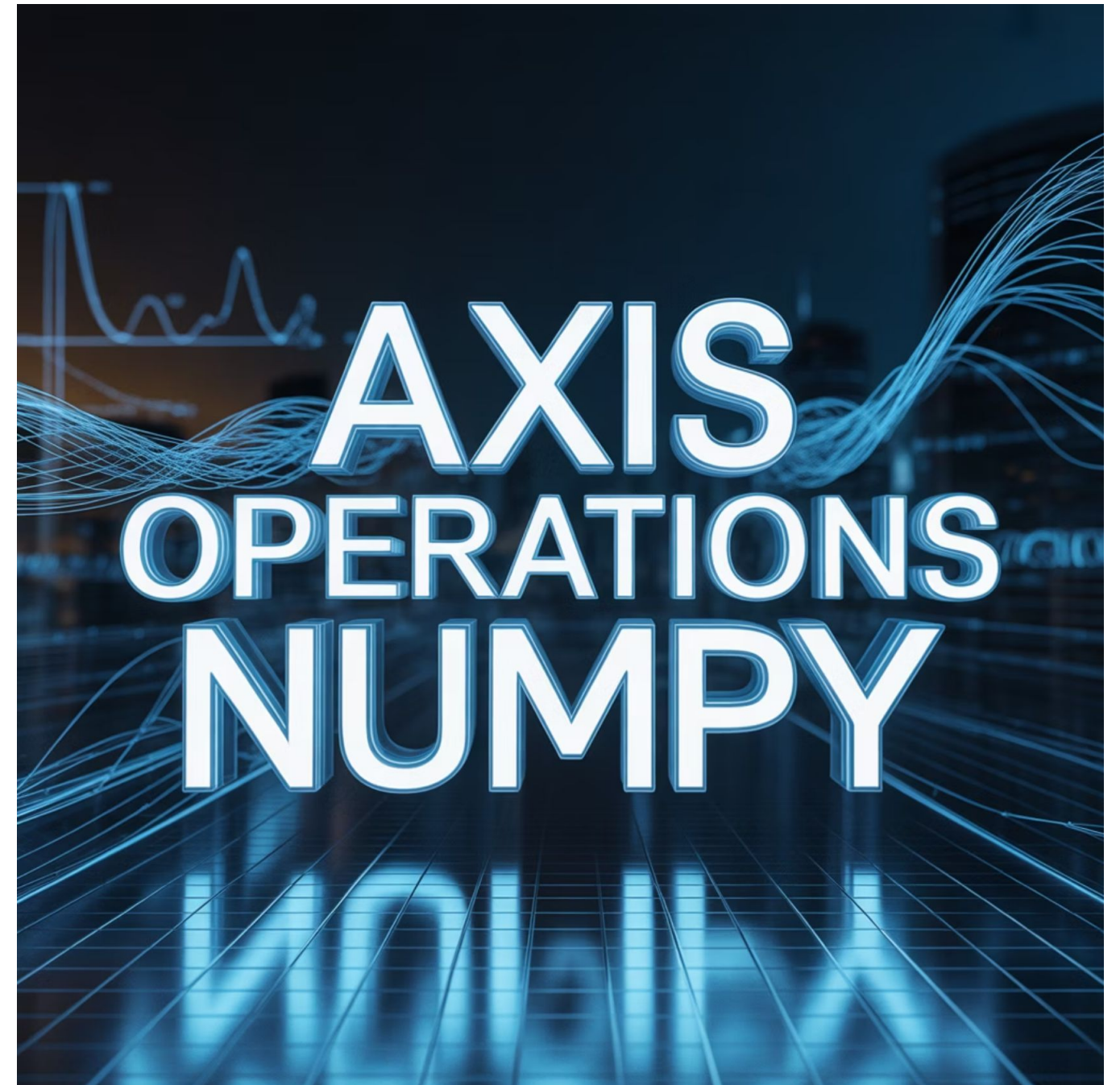
```
valores = np.array([10, 20, 30, 40])
print(np.mean(valores)) # Promedio: 25.0
print(np.std(valores)) # Desviación estándar: 11.18
print(np.min(valores)) # Valor mínimo: 10
print(np.max(valores)) # Valor máximo: 40
print(np.sum(valores)) # Suma total: 100
```

Aplicar funciones sobre matrices

Cuando se trabaja con matrices (2D), NumPy permite aplicar funciones a lo largo de un eje:

```
matriz = np.array([[1, 2, 3],[4, 5, 6]])  
print(np.sum(matriz, axis=0))  
# Suma por columna: [5 7 9]  
print(np.sum(matriz, axis=1))  
# Suma por fila: [6 15]
```

Esto es especialmente útil en análisis por variable (columnas) o por caso (filas).



Funciones booleanas y de comparación

NumPy permite generar arreglos booleanos que indican si se cumple una condición:

```
datos = np.array([3, 7, 2, 9])
print(datos > 5)
# [False True False True]
print(np.any(datos > 5))
# True: al menos un valor cumple
print(np.all(datos > 1))
# True: todos los valores > 1
```

También es posible contar cuántos cumplen la condición:

```
print(np.sum(datos > 5))
# Resultado: 2
```

Funciones universales (ufuncs)

Las funciones matemáticas y estadísticas de NumPy son conocidas como ufuncs o funciones universales. Estas funciones están implementadas en C para operar con alta velocidad sobre cada elemento del arreglo.

Características de las ufuncs:

- Se ejecutan sobre arreglos de cualquier forma o tamaño.
- Admiten operaciones entre arreglos o entre un arreglo y un escalar.
- Se pueden combinar para construir expresiones complejas sin sacrificar velocidad.

Ejemplo:

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(np.add(x, y))
# [5 7 9]
print(np.multiply(x, y))
# [4 10 18]
```

Composición de funciones

NumPy permite combinar funciones en una sola línea para crear pipelines simples:

```
datos = np.array([1, 4, 9, 16])  
resultado = np.log(np.sqrt(datos))  
print(resultado)
```

Esto evita la creación de variables intermedias innecesarias y mejora la legibilidad del código.

Ejemplo: Normalización de datos

Supongamos que recibimos una matriz con mediciones de sensores ambientales y queremos calcular:

- El promedio de cada variable.
- La desviación estándar.
- Un vector con valores normalizados.

Este tipo de transformación es clave en el preprocesamiento de datos para machine learning.

```
datos = np.array([[22.5, 55.1, 83.2],[21.9, 53.2, 80.5],[23.0, 54.8, 81.0]])
media = np.mean(datos, axis=0)
desviacion = np.std(datos, axis=0)
normalizado = (datos - media) / desviacion
print("Normalizado:", normalizado)
```

Actividad guiada: Análisis inicial de lecturas meteorológicas con NumPy



Ejercicio

Análisis inicial de lecturas meteorológicas con NumPy

Objetivo de la actividad

Aplicar técnicas de indexación, selección condicional, operaciones entre arreglos y funciones estadísticas de NumPy, para analizar una matriz de datos meteorológicos simulados.

Situación problema

Formas parte de un equipo técnico que analiza datos recogidos por estaciones meteorológicas automáticas distribuidas en distintas regiones del país. Cada estación registra información horaria sobre tres variables: temperatura (°C), humedad relativa (%) y velocidad del viento (km/h).

Materiales necesarios

- Jupyter Notebook, Google Colab o VSCode con NumPy instalado.



Ejercicio

Instrucciones

Tu jefatura te entrega una matriz de datos correspondiente a 8 horas consecutivas de medición de una estación y te solicita:

- 1 Extraer subconjuntos de interés según condiciones específicas.
- 2 Comparar las mediciones con valores umbral definidos por la política interna de alertas.
- 3 Aplicar funciones estadísticas para obtener indicadores descriptivos.
- 4 Simular condiciones extremas para fines de prueba.

Instrucciones generales

1. Lee con atención el enunciado del problema.
2. Ejecuta el código provisto, paso a paso, observando los resultados obtenidos.
3. Agrega comentarios en el código explicando con tus palabras qué hace cada línea.
4. Responde las preguntas que se plantean al final de cada bloque.
5. Reflexiona sobre cómo podrías adaptar esta lógica a situaciones reales de tu ámbito laboral.



Ejercicio

Preguntas de reflexión

- ¿Qué utilidad práctica tendría la creación de un vector con umbrales en contextos reales?
- ¿Por qué es relevante usar funciones estadísticas como mean, max o min en el análisis de datos?



Resumen



Indexación y slicing

Comprendimos cómo seleccionar elementos dentro de un arreglo NumPy utilizando indexación y slicing, incluyendo técnicas avanzadas.



Selección condicional

Exploramos cómo aplicar condiciones lógicas para seleccionar subconjuntos de datos basados en reglas específicas.



Referencia vs copia

Analizamos las diferencias entre referencia y copia de arreglos, y por qué es importante distinguirlas en operaciones de transformación.



Operaciones vectorizadas

Aprendimos a realizar operaciones entre arreglos de igual forma y con broadcasting, lo que permite cálculos eficientes sin bucles.



Operaciones con escalares

Utilizamos operaciones con escalares para modificar rápidamente todos los elementos de un arreglo.



Funciones universales

Aplicamos funciones matemáticas y estadísticas a arreglos NumPy, incluyendo funciones universales y agregaciones básicas.



Próxima sesión...

Desafío

{desafío}
latam_

